COMPONENTE CURRICULAR:	Desenvolvimento de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

Etapa 1 – Polymorphism: Eliminando Condicionais

a) Identifique o bad smell e o padrão GRASP O *bad smell* presente no método enviar() é o Type Code com Switch/If-Else aninhados (também conhecido como *Long Method* ou *Complex Conditional*). A classe precisa checar manualmente qual é o tipo da notificação em tempo de execução para decidir qual bloco de código rodar. O padrão GRASP indicado para resolver isso é o Polymorphism (Polimorfismo).

b) Aplicação do padrão Polymorphism Para refatorar, devemos criar uma abstração (uma interface ou classe abstrata) e delegar a lógica específica de envio para as subclasses concretas.

Hierarquia:

1. Criamos a interface CanalNotificacao.
2. Criamos as classes concretas NotificacaoEmail e NotificacaoSMS que implementam essa interface.

Pseudocódigo:

Java

```
// Abstração

public interface CanalNotificacao {

    void enviar(String destinatario, String mensagem);

}

// Implementações concretas

public class NotificacaoEmail implements CanalNotificacao {

    @Override
```

```
    public void enviar(String destinatario, String mensagem) {
        // Lógica complexa de e-mail aqui
        System.out.println("Enviando e-mail para " + destinatario);
    }
}
public class NotificacaoSMS implements CanalNotificacao {
    @Override
    public void enviar(String destinatario, String mensagem) {
        // Lógica complexa de SMS aqui
        System.out.println("Enviando SMS para " + destinatario);
    }
}
```

c) Duas vantagens da refatoração

1. Atende ao Princípio Aberto/Fechado (Open/Closed Principle - OCP): Se no futuro precisarmos adicionar WhatsApp ou Push Notification, basta criar uma nova classe NotificacaoWhatsApp que implementa a interface. Não será necessário modificar o código existente, evitando quebrar o que já funciona.
2. Maior coesão e legibilidade: A lógica de envio de e-mail não fica mais misturada com a lógica de SMS em um arquivo gigante. Cada classe cuida exclusivamente das regras do seu próprio canal, facilitando a manutenção e os testes unitários.

Etapa 2 – Pure Fabrication e Indirection: Criando Abstrações Estáveis

a) Pure Fabrication para o Log

- **Classe proposta:** AuditoriaLogger (ou ServicoRegistroNotificacao).
- **Justificativa e Benefício:** Essa classe é uma "Fabricação Pura" porque o conceito de "Log de Auditoria" não existe como um objeto tangível no domínio de negócios principal (que é sobre clientes, finanças e mensagens). É uma

classe inventada por nós, desenvolvedores, para lidar com uma responsabilidade técnica. O benefício é retirar o peso do salvamento de logs das classes de notificação, garantindo que elas tenham apenas um motivo para mudar (enviar a mensagem), mantendo a alta coesão e facilitando a futura migração de arquivo de texto para banco de dados.

b) Indirection para desacoplar a classe Cliente

- **Classe intermediária proposta:** Uma fábrica chamada NotificacaoFactory (ou um despachante como NotificadorService).
- **Como funcionaria:** A classe Cliente não fará mais new NotificacaoEmail(). Em vez disso, ela chama a NotificacaoFactory.criarNotificacao("EMAIL") ou passa a mensagem para um NotificadorService. Essa classe intermediária detém a responsabilidade de instanciar a classe concreta correta, atuando como um "amortecedor" (Indirection) entre o Cliente e as implementações dos canais.

c) Contribuição para Low Coupling e High Cohesion

- **Low Coupling (Baixo Acoplamento):** A *Indirection* garante que a classe Cliente não dependa diretamente (não conheça) as classes de e-mail, SMS ou WhatsApp. O acoplamento do cliente é apenas com a Interface e a Fábrica/Serviço.
- **High Cohesion (Alta Coesão):** A *Pure Fabrication* garante que a lógica de negócio de salvar em banco de dados ou arquivo não polua as classes de notificação. Cada classe faz apenas uma coisa bem feita.

Etapa 3 – Protected Variations e Lei de Demeter

a) Protected Variations no cenário

- **Ponto de variação previsível:** A forma como a auditoria é armazenada (hoje em arquivo de texto, amanhã em banco de dados). Outro ponto é a adição de novos canais de envio.
- **Como aplicar:** Para proteger o sistema da mudança do mecanismo de log, criamos uma interface AuditoriaRepository (com um método registrarLog()). O sistema dependerá dessa interface, e não de uma classe concreta

ArquivoAuditoria. Quando migrarmos para o banco, basta criar uma DatabaseAuditoria que implemente a interface, e o resto do sistema continuará funcionando sem precisar de alterações, pois a variação foi "protegida" pela interface.

b) A Lei de Demeter (Don't Talk to Strangers)

- **Viola a Lei de Demeter? Sim, viola claramente.** O trecho n.getConfiguracao().getServidor().conectar() é um "encadeamento de métodos" (train wreck). A regra da Lei de Demeter diz que um método de um objeto deve invocar apenas métodos do próprio objeto, de seus parâmetros, de objetos que ele mesmo cria ou de seus atributos diretos. A classe Cliente está pedindo a configuração para o objeto Notificacao, depois pedindo o servidor para a Configuracao, e então chamando conectar(). O Cliente está "conversando com os amigos dos amigos" (strangers), conhecendo intimamente a estrutura interna da Notificação.
- **Como reestruturar:** A classe Notificacao deve encapsular esse comportamento. O Cliente deve apenas mandar a notificação fazer o seu trabalho, sem se importar como ela se conecta ao servidor.

Código refatorado:

Java

```
// Método na classe Cliente

public void notificarCliente() {

    // Supondo que 'n' foi obtido via injeção ou fábrica (Indirection)

    CanalNotificacao n = NotificacaoFactory.criar("EMAIL");

    // O cliente apenas manda executar.

    // A conexão com o servidor ocorre internamente na classe de notificação.

    n.enviar(this.email, "Sua fatura chegou!");

}
```